pgBackRest Backup & Recovery

Comprehensive Restore Runbook

PostgreSQL 15 / 16 / 17

Standalone | Streaming Replication (Dual-Node) | Kubernetes (Percona Operator)

Version 1.2 | March 2026

1. Document Overview

This runbook provides step-by-step procedures for every pgBackRest restore scenario across three deployment models: Standalone PostgreSQL, Streaming Replication (Dual-Node), and Kubernetes with Percona Operator. It covers PostgreSQL versions 15, 16, and 17.

1.1 Environment Matrix

Environment	Topology	PGDATA Path	Backup Repo
Standalone	Single PostgreSQL instance	/apps/pgsql_data/17	/apps/backups
Streaming Replication	Primary + Standby (2 nodes)	/apps/pgsql_data/17	/apps/backups
Kubernetes	Percona PG Operator pods	/pgdata/pg<ver>	S3/GCS/repo-host

1.2 Standard Filesystem Layout

The following filesystem layout applies to all Standalone and Dual-Node environments:

Component	Path	Description
PostgreSQL Data Directory	/apps/pgsql_data/17	PGDATA containing all database files
pgBackRest Repository	/apps/backups	Local backup repository (full/diff/incr + WAL)
pgBackRest Logs	/apps/logs	Log files (log-level-file=detail)
WAL Spool Directory	/apps/pgsql_archives/spool	Async archive spool directory
pgBackRest Lock Files	/apps/logs	Lock path for pgBackRest processes
pgBackRest Config	/etc/pgbackrest/pgbackrest.conf	Main pgBackRest configuration file

1.3 pgBackRest Configuration Reference

Standard pgbackrest.conf used across Standalone and Dual-Node environments:

File: /etc/pgbackrest/pgbackrest.conf

```
[stanza]
pg1-path=/apps/pgsql_data/17

[global]
repo1-host-user=postgres
repo1-path=/apps/backups
repo1-retention-full=2
repo1-retention-full-type=count
repo1-retention-history=547
archive-async=y
process-max=2
log-level-file=detail
log-path=/apps/logs
compress=n
archive-push-queue-max=10GB
archive-timeout=60
spool-path=/apps/pgsql_archives/spool
lock-path=/apps/logs

[global:archive-push]
process-max=4
```

Configuration Key Points

- **pg1-path=/apps/pgsql_data/17** — Where pgBackRest finds and restores database files.
- **repo1-path=/apps/backups** — Local repository for all backups and archived WAL.
- **repo1-retention-full=2** (count) — Retains exactly 2 full backup sets.
- **repo1-retention-history=547** — Backup history metadata retained ~18 months.
- **archive-async=y** — Async WAL archiving with spool for better throughput.
- **spool-path=/apps/pgsql_archives/spool** — Must exist and be owned by postgres.
- **compress=n** — Compression disabled.
- **archive-push-queue-max=10GB** — Max spool queue before archiving blocks.
- **process-max=4** (archive-push) — 4 parallel processes for WAL archiving.

> ⚠ **WARNING**
> For Dual-Node setups, both Primary and Standby must have identical pgbackrest.conf with the same stanza name and repo path.

1.4 Backup Types Reference

Backup Type	Description	Restore Impact
Full	Complete copy of all database files	Self-contained; no dependency
Differential	Changes since last full backup	Requires parent full backup
Incremental	Changes since last backup of any type	Requires full chain

1.5 Pre-Restore Checklist

1. Confirm the target recovery point (time, transaction ID, or named restore point).
2. Verify pgBackRest repository is accessible:

```
sudo -u postgres pgbackrest --stanza=stanza info
```

3. Verify disk space (min 2x DB size) on /apps/pgsql_data/17 and /apps/backups:

```
df -h /apps/pgsql_data/17
df -h /apps/backups
```

4. Ensure correct pgBackRest version (2.47+ for PG16, 2.50+ for PG17):

```
pgbackrest version
```

5. Confirm /etc/pgbackrest/pgbackrest.conf stanza name, repo path, pg1-path.
6. Notify stakeholders and obtain change approval.
7. Backup current /apps/pgsql_data/17:

```
sudo -u postgres cp -a /apps/pgsql_data/17 /apps/pgsql_data/17_backup_$(date
+%Y%m%d%H%M)
```

8. Document current WAL position:

```
sudo -u postgres psql -c "SELECT pg_current_wal_lsn(), pg_current_wal_insert_lsn();"
```

9. Verify spool directory: ls -la /apps/pgsql_archives/spool

2. Standalone PostgreSQL Restore Scenarios

All commands use the standard filesystem layout and pgbackrest.conf from Section 1.

2.1 Full Restore to Latest State

Use Case: Complete data loss, hardware failure, or corruption requiring full rebuild.

Prerequisites

- pgBackRest repository at /apps/backups accessible with valid backups
- PostgreSQL 17 binaries installed (matching backup major version)
- Sufficient disk space on /apps/pgsql_data/17 filesystem

Procedure

1. Stop PostgreSQL

```
sudo systemctl stop postgresql-17
```

2. Preserve existing PGDATA and create clean target

```
sudo mv /apps/pgsql_data/17 /apps/pgsql_data/17_old_$(date +%Y%m%d%H%M)
sudo mkdir -p /apps/pgsql_data/17
sudo chown postgres:postgres /apps/pgsql_data/17
sudo chmod 700 /apps/pgsql_data/17
```

3. Execute pgBackRest restore

```
sudo -u postgres pgbackrest --stanza=stanza \
--type=default \
   restore
```

> **ℹ NOTE**
> pgBackRest reads pg1-path from /etc/pgbackrest/pgbackrest.conf and restores files to /apps/pgsql_data/17. --type=default replays all archived WAL from /apps/backups.

4. Start PostgreSQL (enters recovery mode, replays WAL)

```
sudo systemctl start postgresql-17
```

5. Verify recovery completed

```
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
-- Expected: f (recovery complete, server is read-write)

sudo -u postgres psql -c "SELECT pg_postmaster_start_time();"
sudo -u postgres psql -c "SELECT pg_last_xact_replay_timestamp();"
```

6. Validate data integrity

```
sudo -u postgres psql -c "SELECT datname, pg_database_size(datname) FROM
pg_database;"
# Run application-level validation queries
```

7. Verify WAL archiving resumed

```
sudo -u postgres psql -c "SELECT * FROM pg_stat_archiver;"
ls -lt /apps/pgsql_archives/spool/
```

8. Take immediate full backup

```
sudo -u postgres pgbackrest --stanza=stanza --type=full backup
```

2.2 Point-in-Time Recovery (PITR)

Use Case: Recover to a moment before accidental DROP TABLE, bad UPDATE, or corruption.

1. Identify the target recovery time

```
# Check PostgreSQL logs for the event:
sudo grep -i 'drop\|truncate\|delete' /apps/pgsql_data/17/log/postgresql-*.log

# Example: event at 2026-03-05 14:30:00 UTC
# Target: 2026-03-05 14:29:55 UTC
```

2. Stop PostgreSQL and preserve PGDATA

```
sudo systemctl stop postgresql-17
sudo mv /apps/pgsql_data/17 /apps/pgsql_data/17_pre_pitr_$(date +%Y%m%d%H%M)
sudo mkdir -p /apps/pgsql_data/17
sudo chown postgres:postgres /apps/pgsql_data/17
sudo chmod 700 /apps/pgsql_data/17
```

3. Execute PITR restore

```
sudo -u postgres pgbackrest --stanza=stanza \
--type=time \
--target="2026-03-05 14:29:55+00" \
--target-action=promote \
   restore
```

> **i NOTE**
> --target-action: **promote** (writable, default PG12+), **pause** (holds at target), **shutdown** (stops after target).

4. Start and verify

```
sudo systemctl start postgresql-17
sudo -u postgres psql -c "SELECT pg_last_xact_replay_timestamp();"
```

5. Confirm data is intact

```
sudo -u postgres psql -d mydb -c "SELECT count(*) FROM critical_table;"
```

> **⚠ WARNING**
> After PITR, a new timeline is created. Old WAL after the target is inapplicable. Take a new full backup immediately.

6. Take full backup

```
sudo -u postgres pgbackrest --stanza=stanza --type=full backup
```

2.3 Restore a Specific Backup Set

Use Case: Restore an older backup (not the latest).

1. List available backups

```
sudo -u postgres pgbackrest --stanza=stanza info
sudo -u postgres pgbackrest --stanza=stanza info --output=json | python3 -m json.tool
```

2. **Stop PostgreSQL and prepare PGDATA** (same as 2.1, Steps 1-2).

3. **Restore specific set**

```
sudo -u postgres pgbackrest --stanza=stanza \
--set=20260304-120000F \
--type=immediate \
--target-action=promote \
   restore
```

> **i NOTE**
> --type=immediate stops at consistent state (no further WAL). Use --type=default for full WAL replay.

4. **Start, verify, backup**

```
sudo systemctl start postgresql-17
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
sudo -u postgres pgbackrest --stanza=stanza --type=full backup
```

2.4 Selective Database Restore (--db-include)

Use Case: Restore only specific databases from a multi-database cluster.

> **⚠ WARNING**
> pgBackRest restores all files but only included databases are usable. Drop excluded databases after recovery.

1. Stop PostgreSQL and prepare PGDATA (2.1, Steps 1-2).
2. **Restore with filter**

```
sudo -u postgres pgbackrest --stanza=stanza \
--db-include=myapp_production \
--type=default restore
```

3. **Start and drop placeholders**

```
sudo systemctl start postgresql-17
sudo -u postgres psql -c "DROP DATABASE other_db;"
```

2.5 Restore to an Alternate Server

Use Case: Clone for testing, validation, or forensics without touching production.

1. Install PostgreSQL 17 and pgBackRest on target (matching versions).
2. **Configure /etc/pgbackrest/pgbackrest.conf on target** to access the same repo:

```
[stanza]
pg1-path=/apps/pgsql_data/17

[global]
repo1-host-user=postgres
repo1-path=/apps/backups # NFS mount or S3
# For remote: repo1-host=<source-server-ip>
log-path=/apps/logs
spool-path=/apps/pgsql_archives/spool
lock-path=/apps/logs
```

3. **Prepare empty /apps/pgsql_data/17**

```
sudo mkdir -p /apps/pgsql_data/17
sudo chown postgres:postgres /apps/pgsql_data/17
sudo chmod 700 /apps/pgsql_data/17
```

4. **Restore**

```
sudo -u postgres pgbackrest --stanza=stanza \
--type=default --target-action=promote restore
```

5. **Modify postgresql.conf before starting**

```
vi /apps/pgsql_data/17/postgresql.conf

port = 5433
archive_command = '/bin/true'
primary_conninfo = ''
```

6. **Start on alternate server**

```
sudo systemctl start postgresql-17
sudo -u postgres psql -p 5433 -c "SELECT pg_is_in_recovery();"
```

2.6 Tablespace Remap During Restore

Use Case: Original tablespace paths differ on recovery target.

```
sudo -u postgres pgbackrest --stanza=stanza \
--tablespace-map=ts_data=/new/path/ts_data \
--tablespace-map=ts_index=/new/path/ts_index \
   restore
```

2.7 Delta Restore (In-Place Incremental)

Use Case: Limited corruption. Replace only changed files without full PGDATA restore.

1. Stop PostgreSQL

```
sudo systemctl stop postgresql-17
```

2. Delta restore (does NOT require clearing /apps/pgsql_data/17)

```
sudo -u postgres pgbackrest --stanza=stanza \
--delta --type=default restore
```

> **ℹ NOTE**
> Delta compares checksums and replaces only changed files. Much faster for large databases with minor corruption.

3. Start and verify

```
sudo systemctl start postgresql-17
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
```

2.8 PITR to Transaction ID or Named Restore Point

By Transaction ID (XID)

```
sudo -u postgres pgbackrest --stanza=stanza \
--type=xid --target="12345678" \
--target-action=promote restore
```

By Named Restore Point

```
# App creates: SELECT pg_create_restore_point('before_migration_v2');

sudo -u postgres pgbackrest --stanza=stanza \
--type=name --target="before_migration_v2" \
--target-action=promote restore
```

3. Streaming Replication (Dual-Node) Restore Scenarios

Both nodes use the same filesystem layout and pgbackrest.conf from Section 1.

> ⚠ **WARNING**
> Always restore the Primary first, then rebuild the Standby. Never attempt independent restores on both nodes.

3.1 Architecture Reference

Component	Primary	Standby
PGDATA	/apps/pgsql_data/17	/apps/pgsql_data/17
Backup Repo	/apps/backups	/apps/backups (shared)
Logs / Locks	/apps/logs	/apps/logs
WAL Spool	/apps/pgsql_archives/spool	/apps/pgsql_archives/spool
Role	Read-write, archiver	Read-only, receiver

3.2 Primary Node Complete Failure

Decision Matrix

Condition	Action
Standby healthy, data OK	Promote Standby (fastest RTO)
Standby also corrupted/lagging	Restore Primary from pgBackRest, rebuild Standby
Both nodes down	Restore Primary from pgBackRest, rebuild Standby

Option A: Promote Standby

1. **Verify Standby is current**

```
# On standby:
sudo -u postgres psql -c "SELECT pg_last_xact_replay_timestamp();"
sudo -u postgres psql -c "SELECT pg_last_wal_replay_lsn();"
```

2. Promote

```
sudo -u postgres psql -c "SELECT pg_promote();"
# OR: sudo -u postgres pg_ctl promote -D /apps/pgsql_data/17
```

3. Verify and backup

```
sudo -u postgres psql -c "SELECT pg_is_in_recovery();" -- Should be f
sudo -u postgres pgbackrest --stanza=stanza --type=full backup
```

4. Update app connections. Rebuild old Primary as Standby (Section 3.5).

Option B: Restore from pgBackRest

Follow Section 2.1 on Primary, then rebuild Standby per Section 3.5.

3.3 PITR on Primary with Standby Rebuild

Use Case: Accidental data change replicated to Standby. Recover both.

1. Stop BOTH nodes

```
# On BOTH nodes:
sudo systemctl stop postgresql-17
```

2. PITR on Primary (Section 2.2 procedure)

```
sudo mv /apps/pgsql_data/17 /apps/pgsql_data/17_pre_pitr_$(date +%Y%m%d%H%M)
sudo mkdir -p /apps/pgsql_data/17 && sudo chown postgres:postgres /apps/pgsql_data/17
&& sudo chmod 700 /apps/pgsql_data/17

sudo -u postgres pgbackrest --stanza=stanza \
--type=time --target="2026-03-05 14:29:55+00" \
--target-action=promote restore

sudo systemctl start postgresql-17
```

3. Verify, recreate slot, backup

```
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
sudo -u postgres psql -c "SELECT
pg_create_physical_replication_slot('standby_slot');"
sudo -u postgres pgbackrest --stanza=stanza --type=full backup
```

4. Rebuild Standby (Section 3.5).

3.4 Standby Node Failure / Corruption

Use Case: Standby failed or diverged. Primary is healthy.

Option A: Rebuild with pgBackRest (Preferred)

1. **Stop Standby**

```
sudo systemctl stop postgresql-17
```

2. **Delta restore as standby**

```
sudo -u postgres pgbackrest --stanza=stanza \
--type=standby --delta restore
```

3. **Configure primary_conninfo**

```
cat /apps/pgsql_data/17/postgresql.auto.conf | grep primary_conninfo

# If missing:
echo "primary_conninfo = 'host=<primary_ip> port=5432 user=replicator'" \
   >> /apps/pgsql_data/17/postgresql.auto.conf
ls -la /apps/pgsql_data/17/standby.signal
```

4. **Start and verify replication**

```
sudo systemctl start postgresql-17

# On standby:
sudo -u postgres psql -c "SELECT pg_is_in_recovery();" -- t

# On primary:
sudo -u postgres psql -c "SELECT client_addr, state, sent_lsn, replay_lsn,
   pg_wal_lsn_diff(sent_lsn, replay_lsn) AS lag_bytes
   FROM pg_stat_replication;"
```

Option B: pg_basebackup

```
sudo systemctl stop postgresql-17
sudo rm -rf /apps/pgsql_data/17/*
sudo -u postgres pg_basebackup -h <primary_ip> -U replicator \
   -D /apps/pgsql_data/17 -Fp -Xs -P -R
sudo systemctl start postgresql-17
```

3.5 Full Standby Rebuild After Primary Restore

After any Primary restore, the Standby **must** be fully rebuilt (timeline changed).

1. Stop Standby and clear PGDATA

```
sudo systemctl stop postgresql-17
sudo rm -rf /apps/pgsql_data/17/*
```

2. Ensure fresh backup from restored Primary

```
# On primary:
sudo -u postgres pgbackrest --stanza=stanza --type=full backup
```

3. Restore on Standby

```
sudo -u postgres pgbackrest --stanza=stanza --type=standby restore
```

4. Configure primary_conninfo and start (Section 3.4, Steps 3-4).

5. Verify replication caught up

```
# On primary:
sudo -u postgres psql -c "SELECT client_addr,
   pg_wal_lsn_diff(sent_lsn, replay_lsn) AS lag_bytes
   FROM pg_stat_replication;"
-- lag_bytes should be 0
```

3.6 Controlled Switchover (Planned Maintenance)

1. Verify zero replication lag

```
# On primary:
sudo -u postgres psql -c "SELECT pg_wal_lsn_diff(sent_lsn, replay_lsn) AS lag
   FROM pg_stat_replication;"
-- lag MUST be 0
```

2. Stop Primary, promote Standby

```
# On primary:
sudo systemctl stop postgresql-17

# On standby:
sudo -u postgres psql -c "SELECT pg_promote();"
```

3. Convert old Primary to Standby

```
# On old primary:
sudo -u postgres pgbackrest --stanza=stanza \
--type=standby --delta restore

# Set primary_conninfo in /apps/pgsql_data/17/postgresql.auto.conf
sudo systemctl start postgresql-17
```

4. Verify and backup from new Primary

```
sudo -u postgres psql -c "SELECT * FROM pg_stat_replication;"
sudo -u postgres pgbackrest --stanza=stanza --type=full backup
```

4. Kubernetes (Percona PG Operator) Restore Scenarios

> **i NOTE**
> Kubernetes uses operator-managed paths. The pgbackrest.conf from Section 1 does NOT apply. Config is generated from the PerconaPGCluster CR.

4.1 Full Cluster Restore (In-Place)

1. Verify backups

```
kubectl exec -it <cluster>-pgbackrest-repo-host-0 -n <ns> -- \
pgbackrest --stanza=db info
```

2. Apply PerconaPGRestore CR

```
apiVersion: pgv2.percona.com/v2
kind: PerconaPGRestore
metadata:
   name: restore-full-latest
   namespace: <namespace>
spec:
   pgCluster: <cluster-name>
   repoName: repo1
   options:
- --type=default
```

```
kubectl apply -f restore-full.yaml -n <namespace>
```

3. Monitor and verify

```
kubectl get perconapgrestore -n <namespace> -w
kubectl exec -it <cluster>-instance-0 -n <ns> -c database -- \
   psql -U postgres -c "SELECT pg_is_in_recovery();"
```

4.2 PITR on Kubernetes

```
apiVersion: pgv2.percona.com/v2
kind: PerconaPGRestore
metadata:
   name: restore-pitr
spec:
   pgCluster: <cluster-name>
   repoName: repo1
   options:
- --type=time
- --target="2026-03-05 14:29:55+00"
- --target-action=promote
```

4.3 Clone to New Cluster

```
apiVersion: pgv2.percona.com/v2
kind: PerconaPGCluster
metadata:
   name: clone-cluster
   namespace: <new-namespace>
spec:
   postgresVersion: 17
   dataSource:
     pgbackrest:
       stanza: db
       configuration:
       - secret: pgbackrest-secrets
       global:
         repo1-path: /pgbackrest/<original-cluster>/repo1
       repo:
         name: repo1
         s3:
           bucket: my-backup-bucket
           region: us-east-1
```

4.4 Manual Restore (Operator Bypass) — Emergency Only

> ⚠ **WARNING**
> Bypassing the operator causes state drift. Use only as last resort.

1. Scale down operator

```
kubectl scale deployment percona-postgresql-operator --replicas=0 -n <op-ns>
```

2. Exec, stop, restore, start

```
kubectl exec -it <cluster>-instance-0 -n <ns> -c database -- bash
pg_ctl stop -D /pgdata/pg17
pgbackrest --stanza=db --delta --type=default restore
pg_ctl start -D /pgdata/pg17
exit
```

3. Scale operator back

```
kubectl scale deployment percona-postgresql-operator --replicas=1 -n <op-ns>
```

5. Troubleshooting Guide

All diagnostic commands reference the standard paths from Section 1.

5.1 Repository Not Found / Inaccessible

Symptoms: ERROR [055] unable to load info file, ERROR [039] unable to connect to repo

Diagnosis

```
sudo -u postgres pgbackrest --stanza=stanza info
cat /etc/pgbackrest/pgbackrest.conf
ls -la /apps/backups/
ls -la /apps/backups/backup/stanza/
find /apps/backups/backup/stanza/ -name 'backup.info' -ls
```

Resolution

- Fix repo1-path in /etc/pgbackrest/pgbackrest.conf.
- Ensure postgres user has read access to repo directories.
- For remote repos: **sudo -u postgres ssh <repo-host> 'pgbackrest version'**
- After PG upgrade: **pgbackrest --stanza=stanza stanza-upgrade**

5.2 WAL Segment Not Found

Symptoms: ERROR [044] unable to find WAL in archive, recovery halts before target

Diagnosis

```
sudo -u postgres pgbackrest --stanza=stanza info
ls -lt /apps/backups/archive/stanza/ | head -20
ls -la /apps/pgsql_archives/spool/archive/stanza/out/
tail -100 /apps/logs/stanza-archive-push-async.log
sudo -u postgres psql -c "SELECT * FROM pg_stat_archiver;"
```

Resolution

- If WAL purged by retention, PITR target unreachable — restore to latest available.
- Check spool permissions/disk on /apps/pgsql_archives/spool.

- If queue full (10GB), WAL discarded — check 'queue max exceeded' in logs.

5.3 PostgreSQL Version Mismatch

Symptoms: ERROR backup created with PG X.Y but target is A.B

```
sudo -u postgres pgbackrest --stanza=stanza info --output=json | python3 -m json.tool
| grep version
postgres --version
```

- Install matching PG major version. pgBackRest does NOT support cross-major-version restores.

5.4 Insufficient Disk Space

Symptoms: ERROR [040] No space left on device

```
df -h /apps/pgsql_data/17
df -h /apps/backups
df -h /apps/pgsql_archives/spool
sudo -u postgres pgbackrest --stanza=stanza info --output=json | \
   python3 -m json.tool | grep -E 'size|database-size'
```

- Expand filesystem / clean old backups / use --db-include.

5.5 Permission / Ownership Issues

Symptoms: ERROR [041] Permission denied

```
sudo chown -R postgres:postgres /apps/pgsql_data/17
sudo chown -R postgres:postgres /apps/pgsql_archives/spool
sudo chown -R postgres:postgres /apps/logs
sudo chmod 700 /apps/pgsql_data/17
sudo restorecon -Rv /apps/pgsql_data/17
sudo rm -f /apps/logs/stanza-*.lock
```

5.6 PostgreSQL Won't Start After Restore

Corrupted/Missing postgresql.conf

```
cat /apps/pgsql_data/17/postgresql.conf | head -20
initdb --no-locale -D /tmp/fresh_pgdata
cp /tmp/fresh_pgdata/postgresql.conf /apps/pgsql_data/17/
```

Stale PID File

```
rm /apps/pgsql_data/17/postmaster.pid
sudo systemctl start postgresql-17
```

Recovery Signal Issues

```
ls -la /apps/pgsql_data/17/recovery.signal /apps/pgsql_data/17/standby.signal
grep restore_command /apps/pgsql_data/17/postgresql.auto.conf
```

5.7 Replication Not Resuming (Dual-Node)

Diagnosis

```
tail -100 /apps/pgsql_data/17/log/postgresql-*.log
# Look for: connection refused, auth failed, WAL removed, slot missing
```

Checklist

- Verify primary_conninfo in /apps/pgsql_data/17/postgresql.auto.conf
- Check pg_hba.conf: grep replication /apps/pgsql_data/17/pg_hba.conf
- Replication slot: SELECT slot_name, active FROM pg_replication_slots;
- standby.signal exists in /apps/pgsql_data/17/
- Timeline diverged? Fully rebuild Standby (Section 3.5).

5.8 Lock File Conflicts

```
ls -la /apps/logs/stanza-*.lock
ps aux | grep pgbackrest
sudo rm -f /apps/logs/stanza-*.lock
```

5.9 Async Archive Spool Issues

```
du -sh /apps/pgsql_archives/spool/
tail -200 /apps/logs/stanza-archive-push-async.log
ls -lt /apps/pgsql_archives/spool/archive/stanza/out/ | head -20
```

5.10 Kubernetes-Specific

PerconaPGRestore Stuck

```
kubectl describe perconapgrestore <name> -n <ns>
kubectl get events -n <ns> --sort-by=.lastTimestamp | tail -30
```

Restore Pod OOMKilled

```
kubectl describe pod <restore-pod> -n <ns> | grep -A5 'State'
# Fix: spec.backups.pgbackrest.jobs.resources.limits.memory: 4Gi
```

Operator Reconciliation

```
kubectl annotate perconapgcluster <cluster> \
pgv2.percona.com/refresh=$(date +%s) -n <ns>
```

6. PostgreSQL Version-Specific Considerations

Topic	PG 15	PG 16	PG 17
Recovery config	postgresql.conf + signal files	Same as PG 15	Same; target_action=promote
WAL summarizer	N/A	N/A	New: pg_wal_summary
pgBackRest	2.45+	2.47+	2.50+
archive_library	N/A	Introduced	Stable
PGDATA	/apps/pgsql_data/15	/apps/pgsql_data/16	/apps/pgsql_data/17

i NOTE
Mixed PG versions: ensure each server's pgbackrest.conf has the correct pg1-path for its version. Use different stanza names if sharing a repo.

7. Post-Restore Validation Checklist

Check	Command / Action	Expected
Recovery complete	SELECT pg_is_in_recovery();	f (primary)
DBs accessible	\l	All expected DBs
Data integrity	SELECT count(*) FROM critical_table;	Expected counts
Replication	SELECT * FROM pg_stat_replication;	Connected, lag ~0
Extensions	SELECT * FROM pg_extension;	All present
Backup check	pgbackrest --stanza=stanza check	No errors
WAL archiving	SELECT * FROM pg_stat_archiver;	Recent, failed=0
Spool	ls /apps/pgsql_archives/spool/	No stuck WAL
App test	Health check endpoint	HTTP 200
New backup	pgbackrest --stanza=stanza --type=full backup	Success

⚠ WARNING
Always take a new full backup immediately after any restore to establish a clean baseline.

8. Quick Reference

Scenario	Command
Full restore	pgbackrest --stanza=stanza --type=default restore
PITR (time)	pgbackrest --stanza=stanza --type=time --target="..." restore
Specific set	pgbackrest --stanza=stanza --set= --type=immediate restore
Delta	pgbackrest --stanza=stanza --delta restore
As standby	pgbackrest --stanza=stanza --type=standby restore
Delta+standby	pgbackrest --stanza=stanza --type=standby --delta restore
Selective DB	pgbackrest --stanza=stanza --db-include= restore
Tablespace remap	pgbackrest --stanza=stanza --tablespace-map=old=/new restore
PITR (XID)	pgbackrest --stanza=stanza --type=xid --target= restore
PITR (name)	pgbackrest --stanza=stanza --type=name --target= restore
Info	pgbackrest --stanza=stanza info
Verify	pgbackrest --stanza=stanza verify
Stanza upgrade	pgbackrest --stanza=stanza stanza-upgrade

8.1 Filesystem Paths

Path	Purpose
/apps/pgsql_data/17	PostgreSQL data directory
/apps/backups	pgBackRest backup repository
/apps/logs	pgBackRest logs and lock files
/apps/pgsql_archives/spool	Async WAL archive spool
/etc/pgbackrest/pgbackrest.conf	pgBackRest configuration
/apps/pgsql_data/17/postgresql.conf	PostgreSQL main config
/apps/pgsql_data/17/postgresql.auto.conf	Auto-generated config

Path	Purpose
/apps/pgsql_data/17/standby.signal	Standby indicator
/apps/pgsql_data/17/recovery.signal	Recovery indicator

End of Runbook